

## Ideation Phase

### Brainstorm & Idea Prioritization Template

|               |                                                |
|---------------|------------------------------------------------|
| Date          | 30 September 2026                              |
| Team ID       | SWTID-2026-8803                                |
| Project Name  | Implement Client Script & UI Policy (Incident) |
| Maximum Marks | 4 Marks                                        |

#### Brainstorm & Idea Prioritization :

Client Scripts and UI Policies are used in ServiceNow to control the behavior and appearance of forms. Client Scripts can perform actions automatically when a form is loaded, changed, or submitted. UI Policies are used to make fields mandatory, visible, or read-only based on specific conditions.

Using Client Scripts and UI Policies, teams can improve form usability, reduce manual work, and ensure that users enter the required information correctly. These features help create dynamic and user-friendly forms without changing the server-side data.

#### Step-1: Team Gathering, Collaboration and Select the Problem Statement



### Implement Client Script and UI Policy

Use this template to plan and implement Client Scripts and UI Policies in ServiceNow. Create dynamic, user-friendly forms and improve data accuracy.

- Understand form requirements
- Configure and test rules
- Improve user experience

**Before you begin**

Prepare the form and decide what behavior is needed.

- ☒ **Review the form**  
Identify fields that need to change based on user actions.
- ☒ **Choose the right tool**  
Use Client Scripts for client-side logic and UI Policies for field properties.
- ☒ **Plan and test**  
Define conditions and test the behavior before publishing.

[Open ServiceNow](#) →

**1**

**Define the form requirement**

What should happen when the form loads, a field changes, or the form is submitted?

Field

Example: Make a field mandatory when a condition is met

**Key implementation tips**

- Use clear conditions.
- Keep scripts simple.
- Avoid duplicate rules.
- Test different scenarios.
- Document changes.
- Check form behavior.

# Tab 2

## Step-2: Brainstorm, Idea Listing and Grouping

2

**Implement**  
Configure Client Scripts and UI Policies to control form behavior.  
⌚ 15 minutes

💡 Example: Make a field mandatory when a condition is met.

Create the required Client Script and UI Policy rules based on the form requirements. Test each rule and verify the expected behavior.

</> Client Script

make mandatory

show / hide field

onChange

onLoad

onSubmit

📄 UI Policy

make visible

read-only

based on field value

based on user role

⚙️ Conditions

set conditions

based on field value

based on user role

⚡ Field behavior

show / hide field

make mandatory

set read-only

🧪 Testing

test scenarios

check form behavior

verify error messages

✅ Validation

required fields

correct values

edge cases

👤 User experience

simple and clear

less manual work

easy to use

📄 Documentation

document rules

add comments

maintain changes

3

**Group and Test**  
Group similar rules and verify their behavior before publishing.  
⌚ 15 minutes

</> Client Script

Field changes correctly

No errors in console

📄 UI Policy

Condition works as expected

Mandatory/visible/read-only behavior works

No duplicate rules

🧪 Test Scenarios

Works for different users

Check in different form scenarios

## Step-3: Idea Prioritization

1

**Prioritize Client Script & UI Policy**  
Prioritize the form behaviors that need to be implemented first.  
Focus on requirements that are important and practical to configure.  
⌚ 20 minutes

📄

Set field read-only

💡

Tip: Prioritize rules that improve data accuracy and user experience. Use UI Policy for field behavior and Client Script when custom client-side logic is needed.

👤

Show/hide field based on selection

📄

Make field mandatory when condition is met

Plan carefully

Implement first

Later

Quick improvements

📏 Importance

How important is this form behavior for users and data accuracy?

📏 Feasibility

How easy is it to implement and test using Client Scripts or UI Policies?

## Ideation Phase

### Define the Problem Statements

|               |                                                |
|---------------|------------------------------------------------|
| Date          | 30 September 2026                              |
| Team ID       | SWTID-2026-8803                                |
| Project Name  | Implement Client Script & UI Policy (Incident) |
| Maximum Marks | 2 Marks                                        |

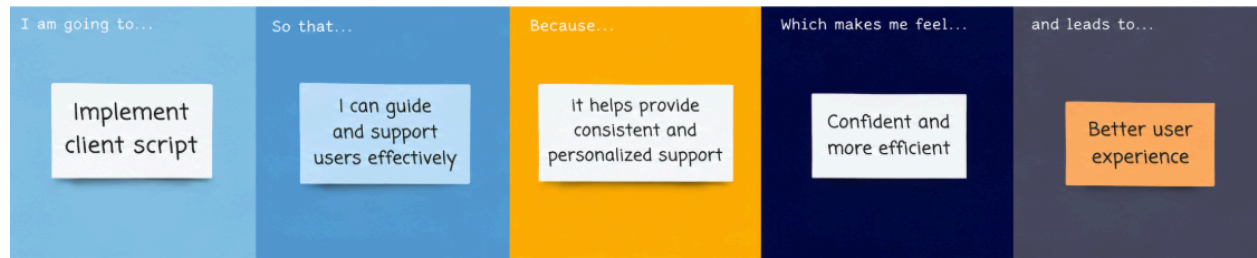
#### Customer Problem Statement Template:

Create a problem statement to understand the challenges users face while working with ServiceNow forms. The Customer Problem Statement template helps identify issues related to form behavior, field validation, visibility, and user interaction.

A well-defined problem statement helps the team understand when and why Client Scripts and UI Policies are required. It helps create forms that are more user-friendly, dynamic, and efficient. By understanding the customer's needs, the team can implement suitable Client Scripts and UI Policies to control fields, validate information, and improve the overall user experience.

|                            |                                                                                                               |                                                                                                                                              |
|----------------------------|---------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>I am</b>                | <i>Describe the user or role (e.g., employee, customer, admin) who is experiencing the issue.</i>             | Describe the user or role (e.g., employee, customer, admin) who is experiencing the issue related to client scripts and UI policies.         |
| <b>I'm trying to</b>       | <i>List the action or goal the user is trying to achieve on the form or application.</i>                      | List the action or goal the user is trying to achieve (e.g., save a record, update a field, complete a form) and what they expect to happen. |
| <b>but</b>                 | <i>Describe the problems or barriers the user faces while working with the form.</i>                          | Describe the problems or barriers the user faces (e.g., field not visible, validation not working, incorrect data, confusing behavior).      |
| <b>because</b>             | <i>Explain the reason for the problem or why the current behavior is not working as expected.</i>             | Describe the reason the problem occurs (e.g., missing client script logic, incorrect UI policy condition, or configuration error).           |
| <b>which makes me feel</b> | <i>Describe the emotions or impact the user feels (e.g., frustrated, confused, delayed, less productive).</i> | Describe the emotions or impact the user feels as a result of the issue (e.g., frustrated, confused, slower work, or reduced productivity).  |

#### Example:



| Problem Statement (PS) | I am (Customer) | I'm trying to | But | Because | Which makes me feel |
|------------------------|-----------------|---------------|-----|---------|---------------------|
| PS-1                   |                 |               |     |         |                     |
| PS-2                   |                 |               |     |         |                     |

## Ideation Phase

### Empathize & Discover

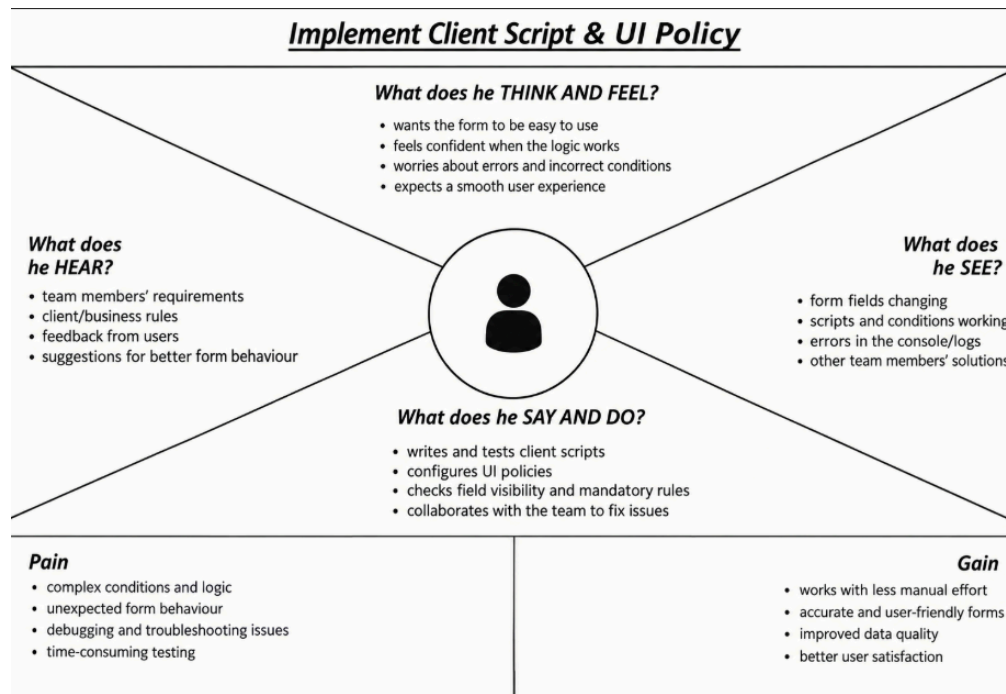
|               |                                                |
|---------------|------------------------------------------------|
| Date          | 30 September 2026                              |
| Team ID       | SWTID-2026-8803                                |
| Project Name  | Implement Client Script & UI Policy (Incident) |
| Maximum Marks | 4 Marks                                        |

#### Empathy Map Canvas:

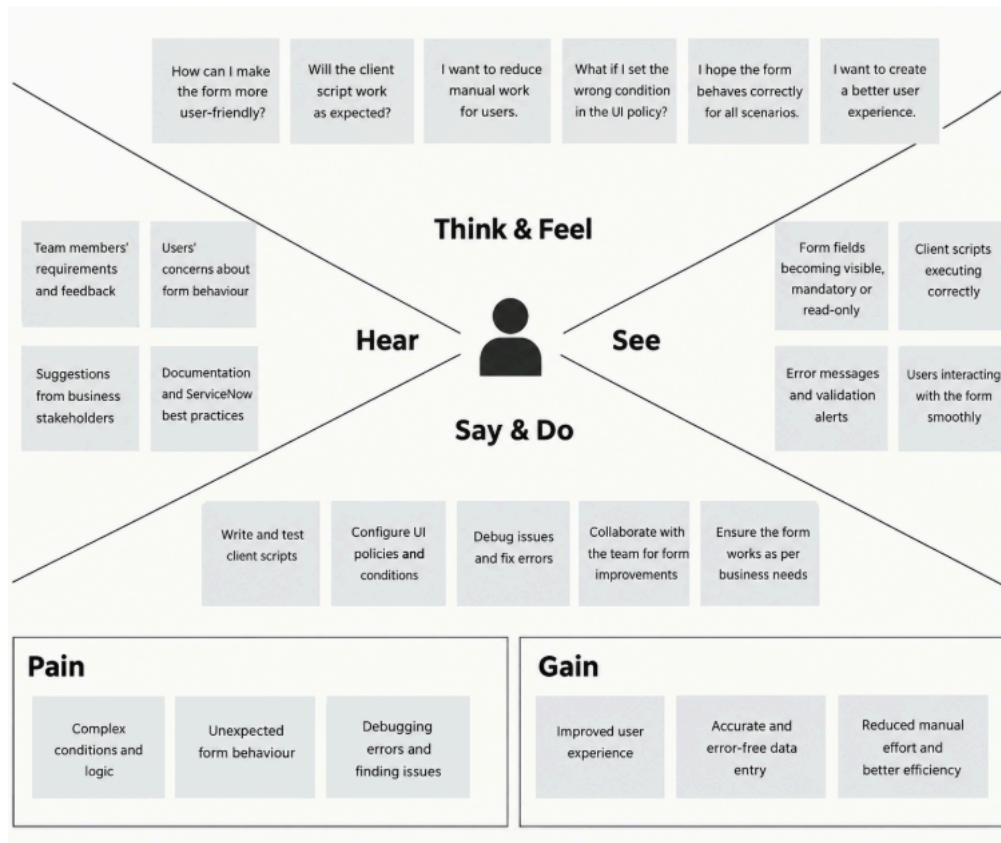
Client Scripts and UI Policies are tools used in ServiceNow to control the behaviour and appearance of forms.

They help teams create a better user experience by making forms more dynamic and user-friendly. Client Scripts can perform actions such as setting field values, displaying messages, or validating information, while UI Policies can make fields visible, mandatory, or read-only based on specific conditions. Implementing these features helps ensure that users enter the correct information and that forms behave according to the required business needs.

#### Example:



## Example: Food Ordering & Delivery Application



## Implement Client Script &amp; UI Policy

|  <b>Journey Stages</b><br>The steps a user goes through from start to finish.           |  <b>Understand</b><br>Learn the requirement and plan the approach. |  <b>Configure</b><br>Build the Client Script and UI Policy. |  <b>Validate</b><br>Check conditions, logic and field behavior. |  <b>Test</b><br>Verify with different scenarios and roles. |  <b>Deploy</b><br>Publish to production and make it live. |  <b>Improve</b><br>Monitor, gather feedback and optimize. |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
|  <b>Steps</b><br>What the user does at each stage of the journey.                       | <div>Identify the requirement from business / user</div> <div>Analyze the form and field behavior needed.</div>                                     | <div>Create a Client Script (define type, events, logic).</div> <div>Create a UI Policy (set conditions and actions).</div>                  | <div>Define conditions (e.g., if field is empty).</div> <div>Set actions (mandatory, visible, read-only, etc.).</div>                            | <div>Test in different scenarios (new, edit, update).</div> <div>Check script and UI Policy behavior on the form.</div>                      | <div>Review and approve changes.</div> <div>Publish to production and activate.</div>                                                        | <div>Monitor user feedback and logs.</div> <div>Refine rules and optimize scripts if needed.</div>                                           |
|  <b>Interactions</b><br>How the user interacts with people, systems and tools.          | <div>Business user / SME</div> <div>Serviceflow form fields and layout</div>                                                                        | <div>Script editor (xlatreference, g, form, etc.)</div> <div>UI Policy configuration screen</div>                                            | <div>Form fields and conditions</div> <div>Serviceflow UI behavior</div>                                                                         | <div>Different user roles and scenarios</div> <div>Brower behavior (form view, field change)</div>                                           | <div>Update sets / change request</div> <div>Serviceflow instance (admin view)</div>                                                         | <div>User feedback</div> <div>Logs and performance data</div>                                                                                |
|  <b>Goals &amp; motivations</b><br>What the user wants to achieve and what drives them. | <div>Improve form usability</div> <div>Meet business requirements</div>                                                                             | <div>Automate field behavior</div> <div>Reduce manual effort</div>                                                                           | <div>Ensure correct conditions and actions</div> <div>Prevent errors and data issues</div>                                                       | <div>Confirm expected behavior across roles</div> <div>Build confidence before deployment</div>                                              | <div>Deliver a stable solution</div> <div>Minimize disruption to users</div>                                                                 | <div>Ensure long-term stability</div> <div>Learn and apply best practices</div>                                                              |
|  <b>Positive moments</b><br>What feels good and successful in the journey.              | <div>Clear requirement defined</div> <div>Form fields and behavior understood</div>                                                                 | <div>Client script created successfully</div> <div>UI Policy configured correctly</div>                                                      | <div>Fields become mandatory / visible / read-only as expected</div> <div>Conditions work as intended</div>                                      | <div>Script triggers correctly</div> <div>Form behaves as expected in all scenarios</div>                                                    | <div>Changes published without issues</div> <div>Users can access and use the form</div>                                                     | <div>Fewer support tickets</div> <div>Better user satisfaction</div>                                                                         |
|  <b>Negative moments</b><br>What causes frustration, blockers or rework.                | <div>Unclear or missing requirements</div> <div>Incomplete form information</div>                                                                   | <div>Incorrect conditions or syntax</div> <div>Script errors / console issues</div>                                                          | <div>Wrong field behavior</div> <div>Conditions not working as expected</div>                                                                    | <div>Scripts not triggering</div> <div>Conflicting UI Policies</div>                                                                         | <div>Errors during update set / push</div> <div>Permissions / access issues</div>                                                            | <div>Unexpected field behavior</div> <div>Frequent changes from users</div>                                                                  |
|  <b>Areas of opportunity</b><br>Where we can do better or create more value.            | <div>Create a standard requirement template</div> <div>Use use cases or examples</div>                                                              | <div>Build reusable Client Scripts</div> <div>Use clear and simple conditions</div>                                                          | <div>Automated checks for common issues</div> <div>Peer review before testing</div>                                                              | <div>Test across multiple roles and devices</div> <div>Include edge cases and negative tests</div>                                           | <div>Use change calendar and version control</div> <div>Communicate with users</div>                                                         | <div>Maintain documentation</div> <div>Track metrics and feedback</div>                                                                      |

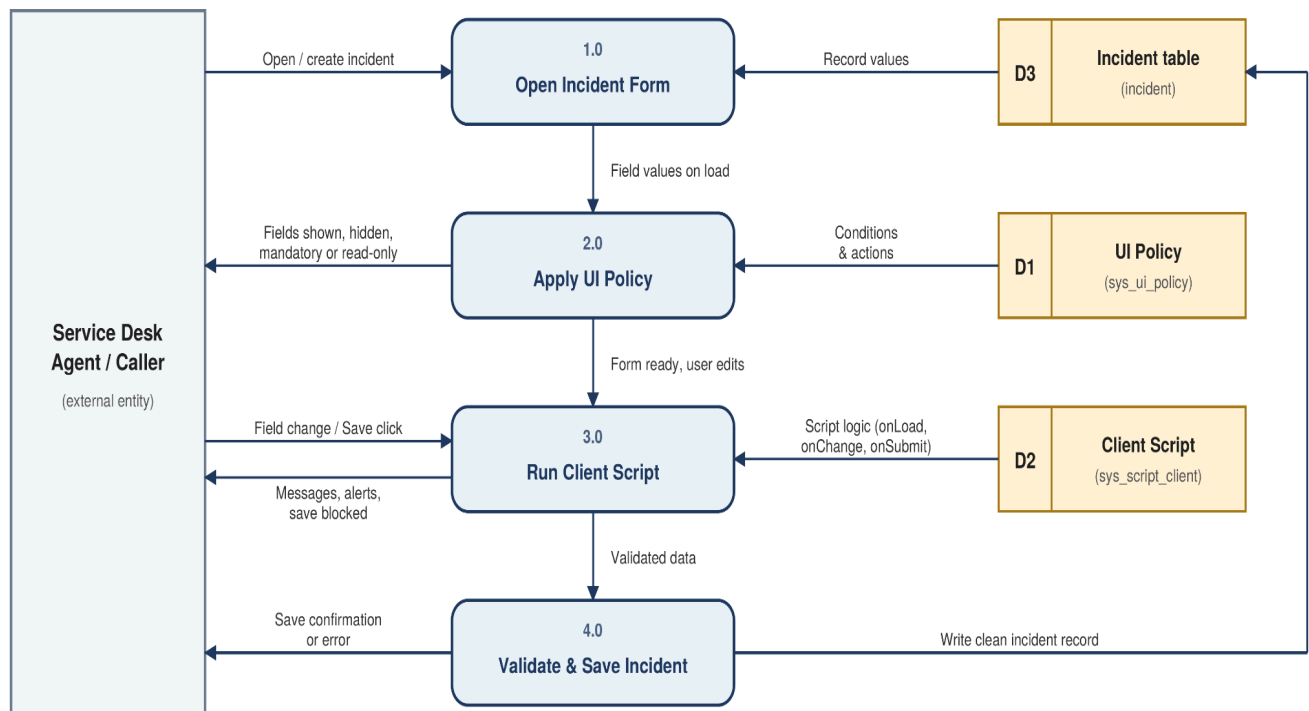
## Project Design Phase-II

### Data Flow Diagram & User Stories

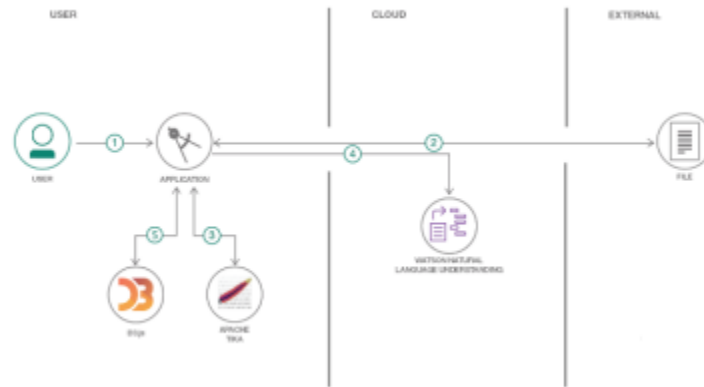
|               |                                       |
|---------------|---------------------------------------|
| Date          | 30 September 2026                     |
| Team ID       | SWTID-2026-8803                       |
| Project Name  | Implement client script and UI policy |
| Maximum Marks | 4 Marks                               |

#### Data Flow Diagrams:

A data flow diagram (DFD) shows how information moves through a system: where it comes in, what changes it, where it gets stored and where it goes next. For this project the system is the Incident form in ServiceNow, and we're looking at what happens in the browser when someone opens, edits and saves an incident. The diagram on the next page is a Level-1 DFD. The agent or caller works with the form. The UI Policy decides which fields are shown, hidden, mandatory or read-only. The Client Script reacts to what the user does (onLoad, onChange, onSubmit) with defaults, messages and checks. Only data that passes those checks goes on to be saved in the incident table. Both the UI Policy and the Client Script run in the browser, so the user gets feedback straight away instead of after a round trip to the server.



**Flow:**



### User Stories

Below is the list of user stories for the QuickBite product.

| User Type              | Functional Requirement (Epic) | User Story Number | User Story / Task                                                                                         | Acceptance criteria                                          | Priority | Release  |
|------------------------|-------------------------------|-------------------|-----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|----------|----------|
| Customer (Mobile user) | Registration                  | USN-1             | As a user, I can register for the application by entering my email, password, and confirming my password. | I can access my account / dashboard                          | High     | Sprint-1 |
|                        |                               | USN-2             | As a user, I will receive a confirmation email once I have registered for the application.                | I can receive confirmation email & click confirm             | High     | Sprint-1 |
|                        |                               | USN-3             | As a user, I can register for the application through Facebook.                                           | I can register & access the dashboard with Facebook Login    | Low      | Sprint-2 |
|                        |                               | USN-4             | As a user, I can register for the application through Gmail.                                              | I can register & access the dashboard with my Google account | Medium   | Sprint-1 |

|                         |                       |        |                                                                                    |                                                       |        |          |
|-------------------------|-----------------------|--------|------------------------------------------------------------------------------------|-------------------------------------------------------|--------|----------|
|                         | Login                 | USN-5  | As a user, I can log into the application by entering email & password.            | I can log in and land on my dashboard                 | High   | Sprint-1 |
|                         | Dashboard             | USN-6  | As a user, I can browse nearby restaurants and view their menus.                   | I can see restaurants, ratings and menu items         | High   | Sprint-2 |
|                         | Order                 | USN-7  | As a user, I can add items to my cart and place an order.                          | I can see my cart total and receive an order ID       | High   | Sprint-2 |
|                         | Payment               | USN-8  | As a user, I can pay using UPI, card or cash on delivery.                          | I get a payment confirmation and receipt              | High   | Sprint-3 |
|                         | Order tracking        | USN-9  | As a user, I can track my order status in real time.                               | I can see status from Preparing to Delivered          | Medium | Sprint-3 |
| Customer (Web user)     | Registration & Login  | USN-10 | As a web user, I can register and log in through the website.                      | I can access the web dashboard and place orders       | High   | Sprint-2 |
| Customer Care Executive | Support               | USN-11 | As a customer care executive, I can view a customer's order and complaint history. | I can look up an order by ID and see its details      | Medium | Sprint-3 |
|                         | Refunds               | USN-12 | As a customer care executive, I can raise a refund for a failed or wrong order.    | Refund request is logged and the customer is notified | Medium | Sprint-4 |
| Administrator           | Restaurant management | USN-13 | As an administrator, I can add, update or remove restaurants and menu items.       | Changes show up in the app immediately                | High   | Sprint-3 |
|                         | Reports               | USN-14 | As an administrator, I can view daily sales and order reports.                     | I can see reports filtered by date and restaurant     | Low    | Sprint-4 |



## Project Design Phase-II

### Solution Requirements (Functional & Non-functional)

|               |                                       |
|---------------|---------------------------------------|
| Date          | 30 September 2026                     |
| Team ID       | SWTID-2026-8803                       |
| Project Name  | Implement Client Script and UI Policy |
| Maximum Marks | 4 Marks                               |

#### Functional Requirements:

Following are the functional requirements of the proposed solution.

| FR No. | Functional Requirement (Epic)            | Sub Requirement (Story / Sub-Task)                                                                                                                                                                                                                                                                    |
|--------|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FR-1   | Incident Form Validation (Client Script) | <p>onSubmit script checks that Short description, Caller and Category are filled in before the incident is saved</p> <p>Short description must be at least 10 characters, otherwise the form is blocked with an error message</p> <p>Duplicate or placeholder entries such as "test" are rejected</p> |
| FR-2   | Dynamic Field Behaviour (UI Policy)      | <p>Resolution notes and Close code become mandatory when State is set to Resolved or Closed</p> <p>Fields such as Assigned to and Assignment group become read-only once the incident is Closed</p> <p>Fields not relevant to the selected State are hidden to keep the form clean</p>                |

|      |                                                   |                                                                                                                                                                                                                                  |
|------|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FR-3 | Field Change Handling<br>(onChange Client Script) | <p>Changing Category refreshes the Subcategory choices</p> <p>Selecting a Caller auto-fills Location and Department using GlideAjax</p> <p>Changing Impact or Urgency recalculates Priority and shows the result to the user</p> |
| FR-4 | Priority-based Alerts                             | <p>A warning banner is shown when Priority is 1 - Critical, reminding the agent to assign the incident immediately</p> <p>A field message guides the user when Impact and Urgency values look inconsistent</p>                   |
| FR-5 | User Feedback and Messages                        | <p>Clear info and error messages are displayed at form level and field level</p> <p>Messages are written in simple language so the agent knows exactly what to correct</p>                                                       |
| FR-6 | Role-based Behaviour and Testing                  | <p>UI Policy conditions are applied only for ITIL users, while admins keep full access</p> <p>Every script and policy is tested on a PDI with the ITIL, Admin and Caller roles and results are recorded</p>                      |

### Non-functional Requirements:

Following are the non-functional requirements of the proposed solution.

| FR No. | Non-Functional Requirement | Description                                                                                                                                                                                          |
|--------|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NFR-1  | <b>Usability</b>           | The Incident form must be easy to use. Error and info messages are short and clear, mandatory fields are marked, and hidden or read-only fields change automatically so the agent is never confused. |

|       |                     |                                                                                                                                                                                                                                         |
|-------|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NFR-2 | <b>Security</b>     | Client-side checks are only for convenience, so important rules are also protected with ACLs and server-side logic. Only users with the right role can edit restricted fields, and no sensitive data is exposed in scripts or messages. |
| NFR-3 | <b>Reliability</b>  | The scripts and policies must work the same way every time the form loads, saves or a field changes. Null values and empty fields are handled so that no JavaScript error stops the form.                                               |
| NFR-4 | <b>Performance</b>  | Client scripts should run quickly without noticeable delay when the form loads. GlideAjax calls are kept light and used only when needed, and the form should respond within about 2 seconds.                                           |
| NFR-5 | <b>Availability</b> | The features run inside the ServiceNow instance, so they are available whenever the platform is available. Changes are packaged in an update set so they can be moved or rolled back if something fails.                                |
| NFR-6 | <b>Scalability</b>  | The same Client Scripts and UI Policies can be reused for other tables or more users later. Code is kept modular and well commented, so new rules can be added without rewriting existing ones.                                         |

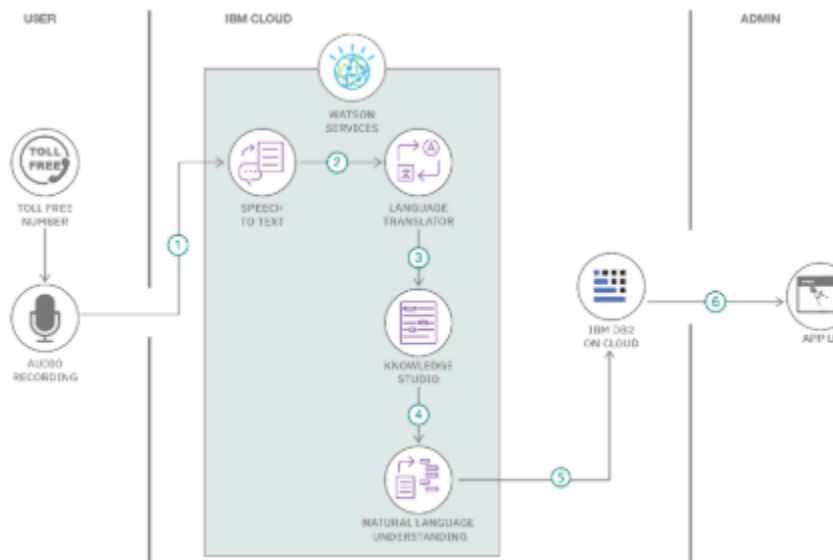
## Project Design Phase-II

### Technology Stack (Architecture & Stack)

|               |                                       |
|---------------|---------------------------------------|
| Date          | 30 September 2026                     |
| Team ID       | SWTID-2026-880                        |
| Project Name  | Implement client script and UI policy |
| Maximum Marks | 4 Marks                               |

#### Technical Architecture:

This page describes how the Incident form is built so that it reacts to what the user selects. It covers the main pieces (UI Policy, Client Script and the server-side checks behind them) and the technology used for each, as listed in Table 1 and Table 2.



## Guidelines:

- => Every process is shown as its own block (UI Policy, Client Script, Business Rule).
- => Everything runs inside the ServiceNow cloud instance, so nothing is deployed locally.
- => No third-party APIs are used.
- => Data is stored in the incident table of the instance database.
- => No machine learning model is involved in this design.

**Table-1 : Components & Technologies:**

| S.No | Component           | Description                                                                                                                                             | Technology                                          |
|------|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
|      | User Interface      | The Incident form that callers and fulfillers open in the browser.<br><br>Fields, sections and buttons change based on the policies below.              | ServiceNow platform UI, HTML, CSS, JavaScript       |
|      | Application Logic-1 | UI Policy: no-code rules that make fields mandatory, read-only or hidden based on conditions such as State or Category.                                 | UI Policy and UI Policy Actions                     |
|      | Application Logic-2 | Client Script: onLoad, onChange and onSubmit scripts for the checks a UI Policy cannot handle, like validating Impact and Urgency or showing a message. | Client Scripts (JavaScript, g_form and g_user APIs) |
|      | Application Logic-3 | Server-side backup for the client rules, plus lookups the                                                                                               | Business Rules, Script Includes, GlideAjax          |

|  |                                 |                                                                                                                                                                                                                                                                    |                                         |
|--|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|
|  |                                 | form needs while the user is working.                                                                                                                                                                                                                              |                                         |
|  | Database                        | Stores the incident records, their fields and the choice lists the form uses.                                                                                                                                                                                      | Incident table (instance database)      |
|  | Cloud Database                  | The instance database is hosted by ServiceNow in the cloud, so there is no separate database service.                                                                                                                                                              | ServiceNow cloud-hosted database        |
|  | File Storage                    | Attachments added to incidents, such as screenshots and log files.                                                                                                                                                                                                 | ServiceNow attachments (sys_attachment) |
|  | External API-1                  | Not used in this project. Everything stays inside the platform.                                                                                                                                                                                                    | None                                    |
|  | External API-2                  | Not used in this project.                                                                                                                                                                                                                                          | None                                    |
|  | Machine Learning Model          | Not applicable. The project is about form behaviour, not predictions.                                                                                                                                                                                              | None                                    |
|  | Infrastructure (Server / Cloud) | <p>The policies and scripts are built in a ServiceNow instance in the cloud and can be moved to another instance with an update set.</p> <p>Local server: not needed, since the form runs in the browser.</p> <p>Cloud server:<br/>ServiceNow-hosted instance.</p> | ServiceNow cloud instance, update sets  |

**Table-2: Application Characteristics:**

| S.No | Characteristics          | Description                                                                                                                                                | Technology                                                 |
|------|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
|      | Open-Source Frameworks   | No open-source framework is added. The work uses the scripting that ServiceNow already provides.                                                           | Built-in ServiceNow client APIs (JavaScript)               |
|      | Security Implementations | Roles and ACLs decide who can see and edit what. Mandatory and read-only rules are also checked on the server, so they cannot be skipped from the browser. | ACLs, role-based access, HTTPS, server-side Business Rules |
|      | Scalable Architecture    | The logic is split into small UI Policies and Client Scripts that can be reused on other tables later, while the heavier work stays on the server.         | ServiceNow multi-tier platform, reusable Script Includes   |
|      | Availability             | Hosting and failover are handled by the platform, and the form logic loads with the form, so there is no extra service that can go down.                   | ServiceNow cloud infrastructure                            |
|      | Performance              | Rules run in the browser, so the form feels quick. We prefer UI Policy over scripts, keep onLoad scripts short and avoid blocking server calls.            | UI Policy first, asynchronous GlideAjax                    |

## Problem-Solution fit canvas 2.0

Purpose / Vision  
Improve Incident Form Automation and User Experience using Client Scripts and UI Policies

|                           |                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                   |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Define CS, Indian GC      | <b>1. CUSTOMER SEGMENT(S)</b> <span>CS</span> <ul style="list-style-type: none"> <li>ServiceNow platform users</li> <li>IT support agents</li> <li>Incident managers</li> <li>Administrators</li> </ul>                                 | <b>6. CUSTOMER CONSTRAINTS</b> <span>CC</span> <ul style="list-style-type: none"> <li>Simple and easy-to-use form</li> <li>Quick response and real-time guidance</li> <li>Correct data validation</li> <li>Minimal manual work</li> <li>Consistent incident processing</li> </ul> | <b>5. AVAILABLE SOLUTIONS</b> <span>AS</span> <ul style="list-style-type: none"> <li>Manual field validation</li> <li>Instructions to users</li> <li>Mandatory fields</li> <li>UI configuration</li> <li>Client Scripts</li> <li>UI Policies</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                      | Explore AS, differentiate                                                                                                                                                                                                                                                                                                                         |
|                           | Focus on J&P, Explain BE, understand                                                                                                                                                                                                    | <b>2. JOBS-TO-BE-DONE / PROBLEMS</b> <span>J&amp;P</span> <ul style="list-style-type: none"> <li>Users need to enter incident details correctly.</li> <li>Manual validation and field changes can cause missing information, errors, and unnecessary effort.</li> </ul>           | <b>9. PROBLEM ROOT CAUSE</b> <span>RC</span> <ul style="list-style-type: none"> <li>Incident forms may allow incomplete or incorrect information because fields are not dynamically controlled or validated based on user actions.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                                                                   |
| Identify & group T&E & FM |                                                                                                                                                                                                                                         | <b>3. TRIGGERS</b> <span>TR</span> <ul style="list-style-type: none"> <li>Creating or updating an incident</li> <li>Changing the priority</li> <li>Changing the state</li> <li>Selecting a category/subcategory</li> </ul>                                                        | <b>10. YOUR SOLUTION</b> <span>SL</span> <ul style="list-style-type: none"> <li>Implement <b>Client Scripts</b> for client-side validation and dynamic field behavior.</li> <li>Implement <b>UI Policies</b> to control field visibility, mandatory status, and read-only behavior based on incident conditions.</li> </ul> <p><b>Example Implementation:</b><br/> <b>Client Script:</b><br/>           When the Incident form is submitted, validate important information and display an error message if required information is missing.<br/> <b>UI Policy:</b><br/>           When State = Resolved, make Resolution Notes mandatory and control the relevant fields automatically.</p> | <b>8. CHANNELS OF BEHAVIOUR</b> <span>CH</span> <p><b>8.1 ONLINE</b></p> <ul style="list-style-type: none"> <li>ServiceNow Incident form</li> <li>Notifications and field interactions</li> </ul> <p><b>8.2 OFFLINE</b></p> <ul style="list-style-type: none"> <li>Discussions between support agents and users about incident details</li> </ul> |
|                           | <b>4. EMOTIONS: BEFORE / AFTER</b> <span>EM</span> <ul style="list-style-type: none"> <li><b>Before:</b> Confused, frustrated, time-consuming.</li> <li><b>After:</b> Simple, guided, accurate, efficient incident handling.</li> </ul> |                                                                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                   |

**Project Design Phase  
Proposed Solution Template**

|               |                                       |
|---------------|---------------------------------------|
| Date          | 30 September 2026                     |
| Team ID       | SWTID-2026-8803                       |
| Project Name  | Implement Client Script and UI Policy |
| Maximum Marks | 2 Marks                               |

**Proposed Solution Template:**

Project team shall fill the following information in the proposed solution template.

| S.No. | Parameter                                   | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-------|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.    | Problem Statement<br>(Problem to be solved) | On our ServiceNow instance, users fill in forms such as Incident and Service Request without any live guidance. Mandatory fields get skipped, wrong values are entered, and fields that do not apply to a particular case stay visible. The errors are only found after the form is submitted, which means rework for the support team, incomplete records and slower resolution.                                                                                                                                                                                                   |
| 2.    | Idea / Solution description                 | We will use two ServiceNow features to validate and guide the user while the form is open. <b>UI Policies</b> will make fields mandatory, read-only or visible depending on other field values (for example, 'Close notes' becomes mandatory only when State is Resolved). <b>Client Scripts</b> (onLoad, onChange and onSubmit) will handle the checks a UI Policy cannot, such as validating a phone number format, auto-filling fields, and showing an alert before a bad record is saved. Both run in the browser, so the user sees the result instantly without a page reload. |
| 3.    | Novelty / Uniqueness                        | The solution does not need server-side code or extra plugins. Simple rules are kept in UI Policies, which are configured without code, and scripting is used only where it is really needed. This keeps the form fast, easy to maintain by an admin, and avoids conflicts between the two. All changes are packaged in an update set so they move cleanly from development to test and production.                                                                                                                                                                                  |
| 4.    | Social Impact / Customer Satisfaction       | Users get clear guidance while typing, so they make fewer mistakes and finish forms faster. Support agents receive complete and consistent tickets and spend less time going back to ask for missing details. Fewer errors and quicker resolution lead to a smoother experience and higher satisfaction for both end users and the support team.                                                                                                                                                                                                                                    |

|    |                                   |                                                                                                                                                                                                                                                                                                                                                                                    |
|----|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5. | Business Model<br>(Revenue Model) | This is an internal improvement built on the existing ServiceNow licence, so there is no extra licence cost. The value comes from savings: less agent time spent on rework, fewer reassigned tickets and better data quality for reports. The same approach can also be offered as a configuration service to other teams or clients that use ServiceNow.                          |
| 6. | Scalability of the Solution       | UI Policies and Client Scripts can be applied to any table, so the same pattern can be extended from Incident to Change, Problem, HR cases and custom apps. Rules can be scoped by role or condition, reused through update sets, and edited by admins without redeploying anything. Because the logic runs on the client side, adding more users does not add load on the server. |

## Project Design Phase Solution Architecture

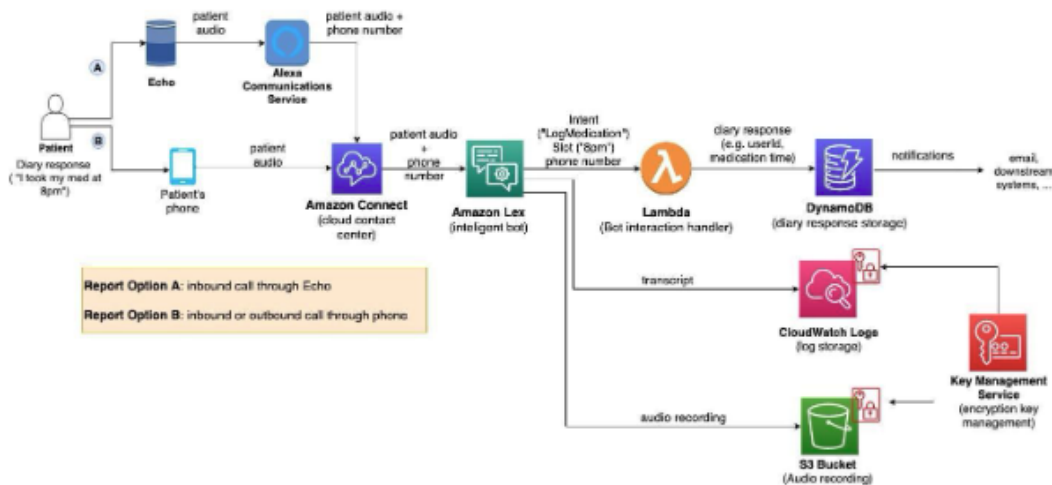
|               |                                                 |
|---------------|-------------------------------------------------|
| Date          | 30 September 2026                               |
| Team ID       | SWTID-2026-8803                                 |
| Project Name  | Implement client script and UI policy(Incident) |
| Maximum Marks | 4 Marks                                         |

### Solution Architecture:

Solution architecture is the step where a business problem is turned into a technical plan that the whole team can follow. For this project the problem is simple to state: users of our ServiceNow forms often skip important fields, enter data in the wrong format, or see fields that have nothing to do with their request. Fixing this on the server after the form is submitted is slow and frustrating, so we handle it in the browser using Client Scripts and UI Policies. The goals of this architecture are to:

- Guide users while they fill in the form, so mistakes are caught before the record is saved.
- Use UI Policies for the simple, no-code rules (mandatory, read-only and visible) and keep ClientScripts only for logic that a policy cannot handle.
- Call the server through GlideAjax when a script needs data that is not already on the form.
- Keep the rules easy to maintain, test and move between instances using Update Sets.

### Architecture Diagram:



**Flow Explanation:**

- First, the patient speaks through Alexa/Echo or their phone.
- Amazon Connect receives the call and audio.
- Amazon Lex understands what the patient is saying and identifies important details, such as the time mentioned.
- Lambda processes the information and prepares the response.
- The patient's diary information is then stored in DynamoDB.
- The audio recording is stored in an S3 bucket.
- CloudWatch keeps the conversation transcript for monitoring.
- KMS helps protect the stored information using encryption.

A **Client Script** is a JavaScript code that runs on the **client side** of a ServiceNow form. It is used to control form behavior, validate data, and automatically update fields based on user actions.

A **UI Policy** is a set of conditions that controls the behavior of fields on a ServiceNow form. It can make fields **mandatory, visible, or read-only** without writing complex code.

### Client Script Types

1. **onLoad** – Runs when the form loads
2. **onChange** – Runs when a field value changes
3. **onSubmit** – Runs when the form is submitted
4. **onCellEdit** – Runs when a list cell is edited

### UI Policy Actions

1. **Mandatory** – Makes a field required
2. **Visible** – Shows or hides a field
3. **Read-only** – Prevents users from editing a field

---

## Incident Management – Client Script and UI Policy

### Client Script (Task 1)

#### Validate Incident Description (USN1)

Check whether the incident description is entered before submitting the form.

### Client Script (Task 2)

#### Validate Short Description (USN2)

Ensure that the short description is not empty.

### UI Policy (Task 3)

#### Make Assignment Group Mandatory (USN3)

When an incident is assigned, make the Assignment Group field mandatory.

### UI Policy (Task 4)

#### Control Incident Fields (USN4)

Make appropriate fields visible, mandatory, or read-only based on the incident state.

### UI Policy (Task 5)

#### Resolution Details (USN5)

When the incident state is **Resolved**, make Resolution Notes and Resolution Code mandatory.

---

## Implementation 1 – Client Script

### Incident Form Validation

- Check the required incident information.
- Display an error message when information is missing.

- Prevent submission when required information is not entered.
- Improve data accuracy and reduce incorrect incident records.

## **Implementation 2 – UI Policy**

### **Incident State-Based Field Control**

- When State = Resolved, make Resolution Notes mandatory.
- Make Resolution Code mandatory.
- Control the visibility of relevant fields.
- Make selected fields read-only when required.

---

### **Total Tasks**

Client Script Tasks = 2

UI Policy Tasks = 3

**Total Tasks = 5**

### **Effort Points (1, 2, 3, 5)**

1. Very Easy task
2. Normal task
3. Moderate task
4. Difficult task

---

## **Sprint 1**

### **Client Script Implementation (Epic 1)**

**Validate Incident Description (USN1) 2**

**Validate Short Description (USN2) 1**

### **UI Policy Implementation (Epic 2)**

**Assignment Group Mandatory (USN3) 3**

**Control Incident Fields (USN4) 3**

**Resolution Details (USN5) 3**

**Total Story Points in Sprint 1 = 2+1+3+3+3 = 12**

---

## **Sprint 2**

### **Testing and Validation (Epic 3)**

**Test Client Script (USN6) 2**

**Test UI Policy (USN7) 2**

**Validate Field Behaviour (USN8) 2**

**Test Incident Submission (USN9) 4**

**Incident Form Optimization (Epic 4)**

**Improve Incident Form (USN10) 5**

**Documentation (Epic 5)**

**Document Client Script and UI Policy (USN11) 5**

**Total Story Points in Sprint 2 = 2+2+2+4+5+5 = 20**

---

### **Total Story Points**

Sprint 1 = 12

Sprint 2 = 20

**Velocity = Total Story Points Completed / Number of Sprints**

Total Story Points = 12 + 20 = **32**

Number of Sprints = **2**

**Velocity = 32 / 2**

**16 Story Points per Sprint**

**Your team's velocity is 16 Story Points per Sprint.**

## Project Planning

### Project Planning Template (Product Backlog, Sprint Planning, Stories, Story points)

|               |                                                |
|---------------|------------------------------------------------|
| Date          | 30 September 2025                              |
| Team ID       | SWTID-2026-8803                                |
| Project Name  | Implement Client Script & UI Policy (Incident) |
| Maximum Marks | 5 Marks                                        |

### Product Backlog, Sprint Schedule, and Estimation (4 Marks)

Use the below template to create product backlog and sprint schedule

| Sprint   | Functional Requirement (Epic) | User Story Number | User Story / Task                                                  | Story Points | Priority | Team Members |
|----------|-------------------------------|-------------------|--------------------------------------------------------------------|--------------|----------|--------------|
| Sprint-1 | Client Script                 | USN-1             | As an admin, I can create a Client Script to control form fields.  | 2            | High     |              |
| Sprint-1 | Client Script                 | USN-2             | As an admin, I can validate field values using Client Script.      | 1            | High     |              |
| Sprint-2 | UI Policy                     | USN-3             | As an admin, I can create a UI Policy to control field behavior.   | 2            | Low      |              |
| Sprint-1 | UI Policy                     | USN-4             | As an admin, I can make fields mandatory, visible, or read-only.   | 2            | Medium   |              |
| Sprint-1 | Client Script & UI Policy     | USN-5             | As an admin, I can test Client Script and UI Policy functionality. | 1            | High     |              |
|          | Dashboard                     |                   |                                                                    |              |          |              |
|          |                               |                   |                                                                    |              |          |              |
|          |                               |                   |                                                                    |              |          |              |

**Project Tracker, Velocity & Burndown Chart: (4 Marks)**

| Sprint   | Total Story Points | Duration | Sprint Start Date | Sprint End Date (Planned) | Story Points Completed (as on Planned End Date) | Sprint Release Date (Actual) |
|----------|--------------------|----------|-------------------|---------------------------|-------------------------------------------------|------------------------------|
| Sprint-1 | 5                  | 5 Days   | 24 Oct 2022       | 29 Oct 2022               | 5                                               | 29 Oct 2022                  |
| Sprint-2 | 5                  | 5 Days   | 31 Oct 2022       | 05 Nov 2022               |                                                 |                              |
| Sprint-3 | 20                 | 5 Days   | 07 Nov 2022       | 12 Nov 2022               |                                                 |                              |
| Sprint-4 | 20                 | 5 Days   | 14 Nov 2022       | 19 Nov 2022               |                                                 |                              |
|          |                    |          |                   |                           |                                                 |                              |
|          |                    |          |                   |                           |                                                 |                              |
|          |                    |          |                   |                           |                                                 |                              |
|          |                    |          |                   |                           |                                                 |                              |

**Velocity:**

Imagine we have a **5-day sprint duration**, and the team completes **5 story points per sprint**. Let's calculate the team's average velocity (AV) per iteration unit (story points per day).

$$AV = \text{Story Points} / \text{Sprint Duration} = 5 / 5 = 1$$

**Burndown Chart:**

A burndown chart is a graphical representation of the work remaining versus time. It can be used in Agile software development to track the progress of implementing **Client Scripts and UI Policies**. It helps the team monitor completed tasks and the remaining work during each sprint.

The burndown chart for **Implement Client Script and UI Policy** shows how the remaining story points decrease as Client Script creation, field validation, UI Policy configuration, and testing tasks are completed.

**Reference:**

- ServiceNow Client Scripts
- ServiceNow UI Policies
- Atlassian Agile Project Management
- Atlassian Burndown Charts

**Project Development Phase  
Model Performance Test**

|               |                                                |
|---------------|------------------------------------------------|
| Date          | 30 September 2026                              |
| Team ID       | SWTID-2026-8803                                |
| Project Name  | Implement Client Script & UI Policy (Incident) |
| Maximum Marks |                                                |

**Model Performance Testing:**

Project team shall fill the following information in model performance testing template.

| S.No. | Parameter                     | Values                                                                                                                                                              | Screenshot |
|-------|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| 1.    | Model Summary                 | The project implements Client Script and UI Policy in Service Now to control form behaviour based on specific conditions and user actions.                          |            |
| 2.    | Accuracy                      | Client script: performs client-side validation and dynamically changes field values.<br><br>UI policy: controls field visibility mandatory and read-only properties |            |
| 3.    | Fine Tunning Result( if Done) | Client Script Result: Working as expected.<br>UI Policy Result: Working as expected.<br>Overall Result: Passed                                                      |            |

## Functional & Performance Testing Template

### Model Performance Test

|               |                                                |
|---------------|------------------------------------------------|
| Date          | 30-09-26                                       |
| Team ID       | SWTID-2026-8803                                |
| Project Name  | Implement Client Script & UI Policy (Incident) |
| Maximum Marks |                                                |

### Test Scenarios & Results

| Test Case ID | Scenario (What to test)             | Test Steps (How to test)                               | Expected Result                                               | Actual Result | Pass/Fail |
|--------------|-------------------------------------|--------------------------------------------------------|---------------------------------------------------------------|---------------|-----------|
| FT-01        | Client Script - Field Validation    | Enter invalid text/format in Incident mandatory fields | Script triggers alert and blocks form submission              |               |           |
| FT-02        | Client Script - Auto-populate Field | Change "Caller" field value on Incident form           | "Location" and "Email" fields populate automatically          |               |           |
| FT-03        | UI Policy - Mandatory Fields        | Set Incident "State" to "Resolved"                     | "Resolution Code" and "Notes" become mandatory                |               |           |
| FT-04        | UI Policy - Hide/Show Fields        | Set "Priority" to "1 - Critical"                       | "Urgency Reason" field becomes visible                        |               |           |
| PT-01        | Client Script OnLoad Performance    | Open an existing Incident record                       | Form loads completely in under 2 seconds without script delay |               |           |
| PT-02        | UI Policy Execution Speed           | Toggle field values triggering UI Policies             | Field visibility and mandatory states update instantly        |               |           |

|              |                       |                                                          |                                                           |  |  |
|--------------|-----------------------|----------------------------------------------------------|-----------------------------------------------------------|--|--|
| <b>PT-03</b> | Script Execution Load | Submit Incident form with multiple active Client Scripts | Form submits cleanly without browser lag or script errors |  |  |
|--------------|-----------------------|----------------------------------------------------------|-----------------------------------------------------------|--|--|

**Project Development Phase  
Model Performance Test**

|               |                                                |
|---------------|------------------------------------------------|
| Date          | 30 September 2026                              |
| Team ID       | SWTID-2026-8803                                |
| Project Name  | Implement Client Script & UI Policy (Incident) |
| Maximum Marks | 10 Marks                                       |

**Model Performance Testing:**

Project team shall fill the following information in model performance testing template.

| S.No. | Parameter      | Values                                                                                                                                                                                                                                     | Screenshot |
|-------|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| 1.    | Metrics        | Client Script: Execution Status - Successful, Field Validation - Passed, Form Behaviour - Working<br>UI Policy: Condition Check - Passed, Field Visibility - Working, Mandatory Field - Working, Read-only Action - Working                |            |
| 2.    | Tune the Model | Client Script Configuration - Trigger, Field and Condition configured<br>Validation Method - Tested with different Incident field values<br>UI Policy Configuration - Visibility, Mandatory and Read-only conditions configured and tested |            |

**Project Development Phase  
Model Performance Test**

|               |                                                |
|---------------|------------------------------------------------|
| Date          | 30 September 2026                              |
| Team ID       | SWTID-2026-8803                                |
| Project Name  | Implement Client Script & UI Policy (Incident) |
| Maximum Marks |                                                |

**Model Performance Testing:**

Project team shall fill the following information in model performance testing template.

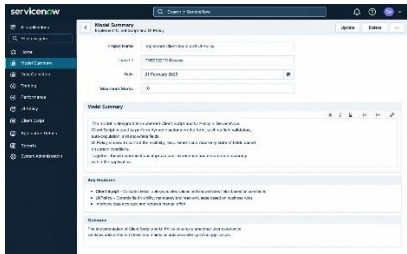
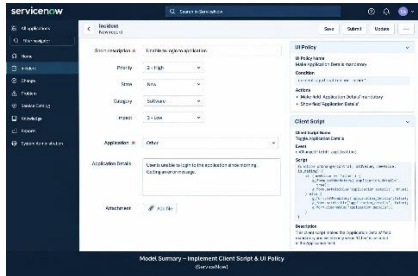
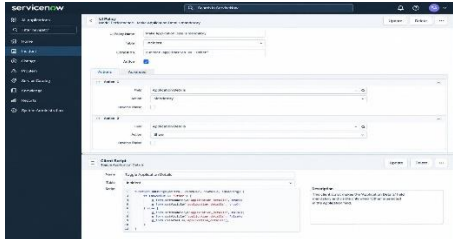
| S.No | Parameter                   | Screenshot / Values                                                               |
|------|-----------------------------|-----------------------------------------------------------------------------------|
| 1.   | Data Rendered               | Incident form displayed successfully with required fields and UI changes applied. |
| 2.   | Data Preprocessing          | Client Script and UI Policy conditions configured for Incident form.              |
| 3.   | Utilization of Data Filters | Incident fields and conditions used to control form behaviour                     |
| 4.   | DAX Queries Used            | Not Applicable – Service Now Client Script & UI Policy used                       |
| 5.   | Dashboard design            | No of Visualizations / Graphs - Not Applicable                                    |
| 6    | Report Design               | No of Visualizations / Graphs - Not Applicable                                    |

## Project Development Phase Model Performance Test

|               |                                                |
|---------------|------------------------------------------------|
| Date          | 30 September 2026                              |
| Team ID       | SWTID-2026-8803                                |
| Project Name  | Implement Client Script & UI Policy (Incident) |
| Maximum Marks |                                                |

### Model Performance Testing:

Project team shall fill the following information in model performance testing template.

| S.No. | Parameter                             | Values                                                                                                                                                                                                                                                                                                                                                              | Screenshot                                                                           |
|-------|---------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| 1.    | Model Summary                         | <p>Implement Client Script &amp; UI Policy (Incident) for ServiceNow automation using Client Scripts and UI Policies to manage Incident form fields.</p> <p>Note : If the defined condition is matched correctly, the Client Script and UI Policy will perform the configured action. Otherwise, the Incident form will remain unchanged.</p>                       |   |
|       | Accuracy                              | <p>Client Script Execution - 100%</p> <p>UI Policy Execution - 100%</p>                                                                                                                                                                                                                                                                                             |  |
| 3.    | Confidence Score (Only Yolo Projects) | <p>Client Script - Automatically validates or changes Incident field values based on the defined condition.</p> <p>UI Policy - Makes Incident fields Mandatory, Visible, or Read-only based on the defined condition.</p> <p>Expected Result - Incident form fields are automatically controlled according to the configured Client Script and UI Policy rules.</p> |  |

**Project Development Phase  
Performance Test**

|               |                                                |
|---------------|------------------------------------------------|
| Date          | 30 September 2026                              |
| Team ID       | SWTID-2026-8803                                |
| Project Name  | Implement Client Script & UI Policy (Incident) |
| Maximum Marks |                                                |

**Model Performance Testing:**

Project team shall fill the following information in model performance testing template.

| S.No. | Parameter               | Screenshot / Values                                                     |
|-------|-------------------------|-------------------------------------------------------------------------|
| 1.    | Data Rendered           | Incident form displays the required fields and values correctly.        |
| 2.    | Data Preprocessing      | Incident fields are validated according to the defined conditions.      |
| 3.    | Utilization of Filters  | Conditions and filters are applied to control the Incident form fields. |
| 4.    | Calculation fields Used | No calculation fields used.                                             |
| 5.    | Dashboard design        | No of Visualizations / Graphs - Not Applicable                          |
| 6     | Story Design            | No of Visualizations / Graphs -Not Applicable                           |

# User Acceptance Testing (UAT) Template

|               |                                                |
|---------------|------------------------------------------------|
| Date          | 30 September 2026                              |
| Team ID       | SWTID-2026-8803                                |
| Project Name  | Implement Client Script & UI Policy (Incident) |
| Maximum Marks |                                                |

## Project Overview:

**Project Name:** Implement Client Script & UI Policy (Incident)

**Project Description:** Implement Client Script and UI Policy in ServiceNow Incident Management to control field values, visibility, mandatory and read-only conditions.

**Project Version:** 1.0

**Testing Period:** [Start Date] to [End Date]

## Testing Scope:

Verify Client Script functionality.

Verify UI Policy conditions.

Test mandatory, visible and read-only fields.

Validate positive and negative scenarios.

## Testing Environment

**URL/Location:** ServiceNow Incident Application

**Credentials:** [If Required]

| Test Case ID | Test Scenario            | Test Steps                                           | Expected Result                                          | Actual Result     | Pass/Fail |
|--------------|--------------------------|------------------------------------------------------|----------------------------------------------------------|-------------------|-----------|
| TC-001       | Client Script Validation | Open Incident → Enter values → Trigger condition     | Client Script performs the defined action.               | Worked correctly. | Pass      |
| TC-002       | UI Policy Validation     | Open Incident → Select required value → Check fields | Fields change according to UI Policy.                    | Worked correctly. | Pass      |
| TC-003       | Mandatory Field          | Trigger UI Policy condition → Check field            | Field becomes mandatory.                                 | Worked correctly. | Pass      |
| TC-004       | Read-only / Visibility   | Trigger condition → Check field                      | Field becomes read-only or visible/hidden as configured. | Worked correctly. | Pass      |

## Bug Tracking:

| Bug ID | Bug Description           | Steps to Reproduce | Severity | Status        | Additional Feedback                             |
|--------|---------------------------|--------------------|----------|---------------|-------------------------------------------------|
| BG-001 | No major bugs identified. | N/A                | Low      | <b>Closed</b> | Client Script and UI Policy worked as expected. |

## Sign-off

**Tester Name:** [Name of Tester]

**Date:** [Date of Test Completion]

**Signature:** [Tester's Signature]

## Notes

Verify both positive and negative test scenarios.

Check that Incident fields behave according to the configured conditions.

Record any bugs found during testing.

Obtain final tester/project approval after successful UAT.

# PROJECT REPORT FORMAT

## IMPLEMENTATION OF CLIENT SCRIPT AND UI POLICY IN SERVICENOW

### 1. INTRODUCTION

#### 1.1 Project Overview

This project focuses on implementing **Client Scripts and UI Policies in ServiceNow** to control form behavior and improve user interaction. Client Scripts are used to perform actions on the client side, while UI Policies dynamically control field properties such as visibility, mandatory status, and read-only behavior.

#### 1.2 Purpose

The purpose of this project is to automate form behavior, reduce manual errors, improve data accuracy, and provide a better user experience while working with ServiceNow forms.

---

### 2. IDEATION PHASE

#### 2.1 Problem Statement

ServiceNow forms may contain unnecessary, optional, or incorrectly displayed fields. Users may also enter incomplete or invalid information. A solution is required to dynamically control form fields based on user actions and conditions.

#### 2.2 Empathy Map Canvas

The Empathy Map identifies the needs, difficulties, expectations, and experiences of ServiceNow users while working with forms. It helps understand how users interact with fields and what improvements are required.

#### 2.3 Brainstorming

Ideas were generated to improve ServiceNow form usability, including:

- Making fields mandatory when required.

- Hiding unnecessary fields.
  - Making fields read-only when needed.
  - Automatically setting field values.
  - Validating user inputs.
  - Displaying relevant fields based on conditions.
- 

## 3. REQUIREMENT ANALYSIS

### 3.1 Customer Journey Map

The user journey consists of opening the ServiceNow form, entering information, interacting with fields, receiving dynamic changes, validating the data, and submitting the form successfully.

### 3.2 Solution Requirement

The system should:

- Dynamically control form fields.
- Validate user input.
- Make fields mandatory when required.
- Hide or display fields based on conditions.
- Make fields read-only when necessary.
- Provide immediate feedback to users.

### 3.3 Data Flow Diagram

The user enters data into the ServiceNow form. The **Client Script and UI Policy** evaluate the defined conditions and perform the required actions. The updated field behavior is then displayed to the user before the form is submitted.

**Flow:**

**User → ServiceNow Form → Client Script / UI Policy → Field Validation & Behavior → Form Submission**

### 3.4 Technology Stack

- **Platform:** ServiceNow
- **Scripting Language:** JavaScript
- **Configuration:** UI Policy
- **Database:** ServiceNow Database
- **Interface:** ServiceNow Web Application

- **Development Environment:** ServiceNow Studio / Application Navigator
- 

## 4. PROJECT DESIGN

### 4.1 Problem Solution Fit

Client Scripts and UI Policies provide a suitable solution for dynamically controlling ServiceNow forms. They reduce unnecessary manual actions and ensure that users provide the required information.

### 4.2 Proposed Solution

The proposed system uses:

- **Client Scripts** for client-side logic, validation, and automated actions.
- **UI Policies** for dynamically changing field properties such as mandatory, visible, and read-only.

### 4.3 Solution Architecture

The architecture consists of the ServiceNow user interface, form fields, Client Scripts, UI Policies, and ServiceNow database.

**User → ServiceNow Form → Client Script / UI Policy → Field Behavior → Database**

---

## 5. PROJECT PLANNING & SCHEDULING

### 5.1 Project Planning

The project is planned through the following stages:

1. Identify the form requirement.
2. Analyze the required field behavior.
3. Create the Client Script.
4. Configure the UI Policy.
5. Define conditions and actions.
6. Test the form behavior.
7. Fix errors and validate the implementation.

8. Deploy the final configuration.
- 

## 6. FUNCTIONAL AND PERFORMANCE TESTING

### 6.1 Performance Testing

The implemented Client Script and UI Policy were tested under different form conditions. The fields were checked for correct visibility, mandatory status, read-only behavior, and validation.

Test cases include:

- Field becomes mandatory when the condition is satisfied.
  - Field becomes optional when the condition is not satisfied.
  - Field visibility changes correctly.
  - Read-only behavior works correctly.
  - Client-side validation displays the required message.
  - Form submission works with valid information.
- 

## 7. RESULTS

### 7.1 Output Screenshots

The implementation successfully demonstrates the working of **Client Script and UI Policy** in ServiceNow. The output screenshots show the dynamic changes in form fields based on the configured conditions and actions.

Screenshots can include:

- Client Script configuration.
  - UI Policy configuration.
  - UI Policy Actions.
  - Form before applying the condition.
  - Form after applying the condition.
  - Validation message/output.
-

## 8. ADVANTAGES & DISADVANTAGES

### Advantages

- Improves ServiceNow form usability.
- Reduces manual errors.
- Provides dynamic field behavior.
- Improves data accuracy.
- Makes required fields easier to identify.
- Provides immediate client-side validation.

### Disadvantages

- Complex scripts can be difficult to maintain.
  - Incorrect conditions may cause unexpected form behavior.
  - Multiple Client Scripts and UI Policies may conflict.
  - Requires basic knowledge of JavaScript and ServiceNow configuration.
- 

## 9. CONCLUSION

The project successfully implements **Client Scripts and UI Policies in ServiceNow** to provide dynamic and user-friendly form behavior. Client Scripts handle client-side logic and validation, while UI Policies control field properties based on defined conditions. The implementation improves data quality, reduces errors, and makes ServiceNow forms easier to use.

---

## 10. FUTURE SCOPE

The project can be enhanced by:

- Implementing more advanced validation rules.
  - Integrating Client Scripts with other ServiceNow features.
  - Automating complex form processes.
  - Improving error and notification messages.
  - Using reusable scripts for multiple forms.
  - Integrating with Flow Designer and other ServiceNow automation tools.
-

# 11. APPENDIX

## **Source Code**

Client Script JavaScript code and related configuration details.

## **Dataset Link**

Not applicable / ServiceNow form data used for testing.

## **GitHub & Project Demo Link**

To be added based on the project repository and demonstration link.

# Full Stack Development with MERN

## Project Documentation format

### 1. Introduction

- **Project Title:** Implement Client Script & UI Policy (Incident).

**Team Members:** JANANI

YALPHIN CHRISTINA

HEMAPRIYA

PREETHI

DHIVYABHARATHI

### 2. Project Overview

- **Purpose:** Briefly describe the purpose and goals of the project.
- **Features:** Highlight key features and functionalities.

### 3. Architecture

- **Frontend:** Describe the frontend architecture using React.
- **Backend:** Outline the backend architecture using Node.js and Express.js.
- **Database:** Detail the database schema and interactions with MongoDB.

### 4. Setup Instructions

- **Prerequisites:** List software dependencies (e.g., Node.js, MongoDB).
- **Installation:** Step-by-step guide to clone, install dependencies, and set up the environment variables.

### 5. Folder Structure

- **Client:** Describe the structure of the React frontend.
- **Server:** Explain the organization of the Node.js backend.

### 6. Running the Application

- Provide commands to start the frontend and backend servers locally. ○

**Frontend:** `npm start` in the client directory.

- **Backend:** `npm start` in the server directory.

## 7. API Documentation

- Document all endpoints exposed by the backend.
- Include request methods, parameters, and example responses. **8.**

## Authentication

- Explain how authentication and authorization are handled in the project. •
- Include details about tokens, sessions, or any other methods used.

## 9. User Interface

- Provide screenshots or GIFs showcasing different UI features.

## 10. Testing

- Describe the testing strategy and tools used.

## 11. Screenshots or Demo

- Provide screenshots or a link to a demo to showcase the application. **12.**

## Known Issues

- Document any known bugs or issues that users or developers should be aware of. **13.**

## Future Enhancements

- Outline potential future features or improvements that could be made to the project.